面向银河麒麟端侧 OS Agent 的多源融合偏好与知识记忆优化方案

技术报告编号 XA-202612-TR-01

摘要

本报告面向银河麒麟桌面操作系统 V11 上的端侧 OS Agent，实现了一套记忆优化方案，覆盖多源数据整合、偏好记忆动态捕捉、知识记忆结构化整合、新旧知识冲突处理与关联检索优化五个环节。方案全栈调用官方组件：文本向量化使用银河麒麟 KYSDK Embedding（768 维），向量存储默认使用官方 kylin-ai-vector-engine，另外实现一套接口等价、可一键切换的轻量 numpy 后端，用于量化论证端侧轻量化程度。知识层不以单条工具调用为入库单位，历史案例、工作流程与可复用模板这三类操作性知识都从事件之间的关系中挖掘。偏好提取采用自监督评测口径，用前期数据预测留出期的真实行为，标准答案直接取自留出期的行为记录，不依赖人工标注。知识冲突的处理策略是给旧知识标注适用边界并保留可回溯的旧版本，旧知识不被新知识覆盖；结构挖掘、语义解释与人工裁决分属三条独立代码路径，被测系统因此无法自产标准答案。在自建的 227 题冻结评测集与 86 条真实知识单元语料上实测：知识检索 recall@10 = 86.3%（Wilson 95% 置信区间下界 81.3%），最难一档（hard）的 recall@1 为 32.9%，同期纯词面基线仅 2.6%，领先 30.3 个百分点；偏好提取在操作习惯类的留出期预测准确率为 90.0%（置信区间下界 88.3%），相对全局众数基线提升 69.5 个百分点；知识冲突判定准

确率在全部标注数据上为 91/96 = 94.8%（置信区间下界 88.4%，达标），其中留出集单独为 56/60 = 93.3%（下界 84.1%）——规则于该批数据产生之前即已冻结；端到端检索响应 P95 为 111.70 ms，占 500 ms 预算的 22.3%。两个正交对照实验支撑“能力来自于自机制设计”这一论点：仅替换生成模型时，记忆机制在 2B/4B/9B 端侧模型与云端参照模型四档上均产生正增益，该结论在三种判分口径下全部成立；仅替换向量化模型时，赛题指定的 KYSDK Embedding 与 BGE-M3 dense 在 R@10 上基本持平（差 2 题），在 R@3、R@5 上指定接口反而各多命中 12~14 题。配套的分层可复现体系使 145 条数字主张可机器核对（L1 115 / L2 30，条数以《可复现性体系/当前数字看板.md》为准），其中约八成不依赖银河麒麟环境，任何人可在普通机器上用固化向量一键复算。四项硬指标中没有在统计意义上达标的部分，本报告同时给出点估计与区间下界的差距并给出优化方向。

目　　录

一、研究动机 5
二、相关工作 6
1. 检索增强与关联检索策略 6
2. 公开数据集考察：为何必须自建评测集 7
三、系统设计 8
1. 总体架构 8
2. 多源数据整合（对应赛题要求 (1)） 9
3. 知识记忆结构化整合（对应赛题要求 (3) 前半） 11
4. 偏好记忆动态捕捉（对应赛题要求 (2)） 12
5. 新旧知识冲突处理（对应赛题要求 (3) 后半） 13
6. 关联检索优化与端侧存储（对应赛题要求 (4)） 15
7. 自然语言指令驱动的精准遗忘（对应赛题要求 (5) 后半） 16
四、评测与结果 18
1. 实验环境 18
2. 评测方法学 19
3. 知识检索召回率（硬指标：≥85%） 19
4. 关联检索消融 22
5. 偏好提取准确率（硬指标：≥85%） 24
6. 知识冲突处理正确率（硬指标：≥88%） 25
7. 检索响应延迟与端侧轻量化（硬指标：≤500 ms） 30
8. 硬指标汇总 32
9. 模型消融 Benchmark：记忆机制是否依赖生成模型规模 34
10. BGE-M3 对照：赛题指定的向量化接口是否构成短板 35
11. 精准遗忘：定位、生效与副作用 37
五、可复现性体系 40
六、结论、局限与后续工作 42
七、安装部署指南 45
1. 环境要求 46
2. 系统解锁与 KYSDK 部署 46
3. 一个必须先知道的限制：这套部署不会持久 47

4. 构建知识索引 48
5. 启动记忆服务 49
6. 验证安装 49
八、操作说明..... 49
1. 知识检索接口 49
2. 偏好查询与冲突回溯 50
3. 精准遗忘 51
4. 接入 OS Agent：pi 记忆钩子 52
5. 评测复现流程 52
参考文献..... 53
附录 A：交付物与代码位置对照..... 56
附录 B：初版待更新数字的结清说明..... 58

一、研究动机

OS Agent 的记忆系统由两大核心能力构成，分别为偏好记忆与知识记忆。其中，偏好记忆服务于用户个性化适配，可根据用户使用习惯定制差异化服务体验；知识记忆主要负责沉淀全场景使用经验与有效信息，实现历史能力与数据的高效复用，提升智能体响应效率。

记忆系统的数据覆盖多维度来源，包含工具运行输出结果、用户自定义配置、全场景操作行为日志等。但多源异构的数据特性，使得数据采集、加工、存储全链路存在诸多痛点，直接制约了记忆模块的整体落地效果。

在实际落地过程中，核心问题集中在三大维度：一是工具输出数据结构不规范、数据质量波动大，造成用户偏好识别精准度偏低，知识沉淀效率与标准化程度不足；二是用户行为数据采集存在缺失，跨场景数据口径不统一，无法完整还原用户真实使用逻辑；三是系统配置版本零散混乱，缺乏规范化的版本管控机制。

上述问题相互叠加，最终形成四大核心技术短板：用户动态偏好感知存在偏差、配置版本运维成本高、新旧知识迭代冲突无法主动甄别与消解、记忆关联检索的精准度不足。

针对行业及现有方案的现存痛点，本次研究完成了多项关键技术突破与体系搭建，核心创新成果如下：

第一，构建**端侧 OS Agent 多源融合记忆架构**。方案全栈适配银河麒麟官方向量嵌入 SDK 与原生向量引擎，同时搭建轻量 Numpy 后端对照实验，完成多组量化对比测试，充分验证了整套方案的性能优势与端侧适配能力。

第二，优化知识挖掘机制与偏好评测范式。在知识挖掘层面，基于场景事件的关联逻辑，解构提炼三类可直接落地应用的实操性结构化知识；在评测层面，摒弃传统低效的人工标注方案，依托历史存量行为数据完成用户偏好预测，结合用户后续真实行为校验模型准确率，打造出贴合真实业务场景、高效低成本的自动化评测体系。

第三，建立可控的知识迭代与冲突治理机制。通过明确各类知识的适用边界，实现新知识更新与旧有效知识留存的双向兼容，杜绝有效信息被盲目覆盖；同时将结构挖掘、语义解析、人工裁决三大核心能力进行链

路解耦，拆分为独立代码分支，大幅简化系统逻辑，提升后续迭代、优化与运维的灵活性。

第四，搭建**分层可复现的标准化评测体系**。本次设计双层校验机制：L1层支持无麒麟环境快速自动化测试，适配日常迭代验证；L2层依托真机环境完成高精度复测核验。本次工作累计实现145项结论的机器自动化核验，双层分级标准及详细核验条目可查阅《可复现性体系/当前数字看板.md》。同时，针对四项核心性能指标，同步输出点估计结果与 Wilson 95%置信区间，确保实验数据客观严谨、结论可信。

本次报告整体章节布局层层递进、逻辑闭环：第二章梳理国内外相关研究现状，汇总公开数据集调研成果，明确现有技术基线与研究空白；第三章详细阐述整体系统架构，拆解五大核心模块的设计思路与技术细节；第四章系统性介绍评测方案，公示四项核心指标的实测性能与实验结论；第五章完整拆解分层可复现评测体系的搭建逻辑、技术路径与落地细节；第六章总结全文核心工作，客观分析当前方案的局限性，并明确后续迭代优化的核心方向。

二、相关工作

1.检索增强与关联检索策略

本方案的知识记忆层级，整体依托检索增强生成（RAG）的核心范式搭建，核心思路是将模型内置的参数化知识，与外部可动态检索的结构化知识做解耦隔离，以此保障知识更新的灵活性与内容的准确性。在检索能力的落地优化上，我们完成了三类经典检索策略的工程实现与消融对比验证，分别是适配关键词匹配的 BM25稀疏字面检索算法、主打检索结果多样性的 MMR 最大边缘相关重排策略，以及依托知识类型聚类质心设计的软先验检索机制，通过消融实验明确了各类策略的实际增益与适用场景。向量存储层面，本方案采用银河麒麟官方迭代的 kylin-ai-vector-engine 1.2.0.1引擎，该引擎基于 Milvus Lite 深度定制优化，为麒麟端侧场景专属分支，整体服务调用基于 Milvus C++ 原生客户端接口开发，适配性与运行性能更贴合端侧设备特性。

向量化能力严格遵循赛题规范，主体业务链路统一调用银河麒麟 KYSDK Embedding 官方模型完成向量编码。同时依据官方答疑许可，我

们引入开源 BGE-M3模型作为对照实验组，用于量化验证自研方案的性能优势，相关对比实验数据与详细分析结果，统一收录于报告4.10章节。

2.公开数据集考察：为何必须自建评测集

本方案的知识记忆层级，整体依托检索增强生成（RAG）的核心范式搭建，核心思路是将模型内置的参数化知识，与外部可动态检索的结构化知识做解耦隔离，以此保障知识更新的灵活性与内容的准确性。在检索能力的落地优化上，我们完成了三类经典检索策略的工程实现与消融对比验证，分别是适配关键词匹配的 BM25稀疏字面检索算法、主打检索结果多样性的 MMR 最大边缘相关重排策略，以及依托知识类型聚类质心设计的软先验检索机制，通过消融实验明确了各类策略的实际增益与适用场景。

本课题聚焦操作系统层级的智能体记忆能力评测，核心需要覆盖三大核心特征：用户跨会话行为延续轨迹、长期跨天的使用状态波动、知识迭代更新与版本更替的关联关系。为筛选适配课题场景的基准数据，本项目对8类主流公开数据集开展逐项核验与适配性研判，完整核查记录与原文引用详见《公开数据集考察记录.md》。

经调研发现，现有主流公开数据集均无法匹配本课题的评测诉求。其中，MultiWOZ、PersonaChat、DailyDialog 等经典对话数据集，以及 TREC Deep Learning Track、MS MARCO 等主流检索评测数据集，均为静态固化标注数据，不具备跨会话关联、时间迭代、版本更替的动态维度。两套同名 ToolBench 数据集也存在明显短板，数据分别依托 ChatGPT 合成指令与人工模板构建，缺失真实用户长期行为与知识迭代特征。而 SAP-SAM 等 BPMN 业务流程数据集，虽具备用户关联、时间戳、版本修订等字段，但聚焦业务流程建模场景，并非操作系统实操场景，与本课题研究范畴不符。

同时本次调研明确两处关键边界限定，避免认知偏差：其一，TREC 系列数据集虽设有时间时效评测赛道，但其时效性仅针对外部公共事件的时序演变，和本课题核心研究的「新旧知识迭代替代、知识冲突更新」语义完全不同；其二，PersonaChat 数据集依托固定人设复用实现多轮对话弱关联，仅为文本层面的隐性关联，并非本课题所需的结构化、可追溯的跨会话行为轨迹，因此无法直接复用。

基于全面调研能够确定，当前无适配本课题 OS 端侧记忆评测场景的公开数据集，因此本项目采用**自建评测数据集**，公开数据集仅作为算法方法研究的参考依据。针对自建数据集无法被第三方复采的固有短板，我们通过第五章搭建的分层可复现性体系，完整补齐数据可核验、可复现能力，保障评测结果严谨可信。

三、系统设计

1. 总体架构

系统分为四层，数据自下而上流动（图3.1）。

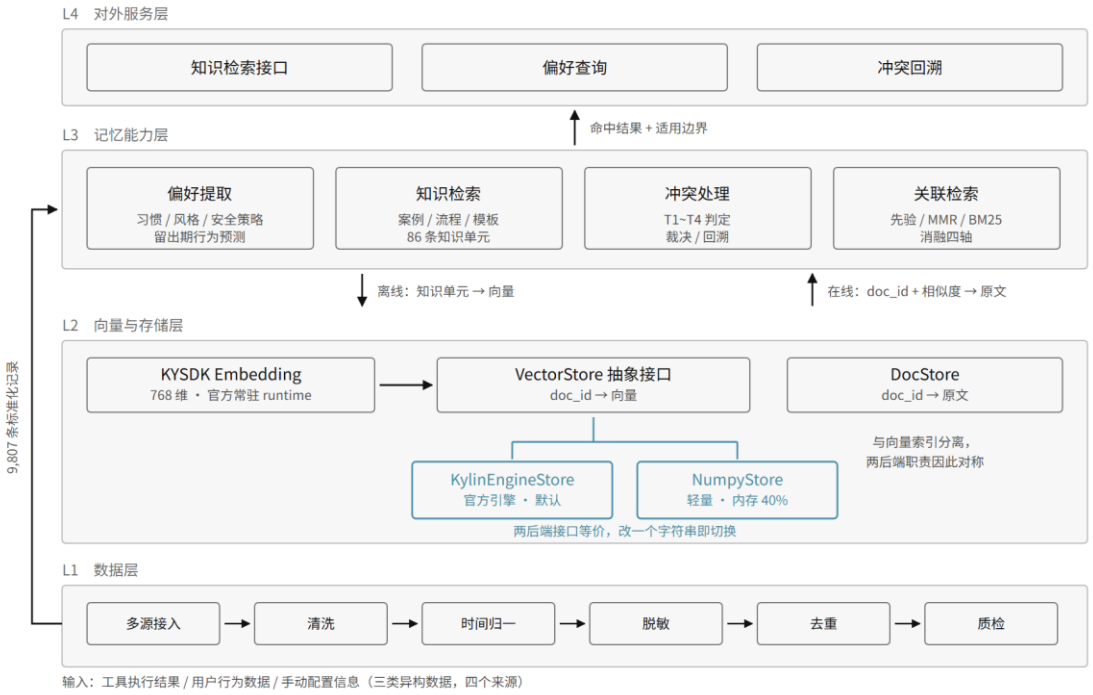


图3.1 系统总体架构

在整体架构的数据通路上，本课题做了清晰的层级解耦与链路优化：数据层完成标准化结构化沉淀后，可绕过向量服务层，直接向上输送至记忆能力层的知识挖掘模块，实现原始有效数据的高效复用。整体设计中，向量层仅作为被动调用的基础服务能力，没有参与核心数据流转主通路，最大限度精简链路开销。

记忆能力层与向量层之间构建了双向异步交互通路，适配离线构建、在线推理双阶段业务逻辑：离线阶段，记忆层将规整后的知识单元推送至向量层完成索引构建；在线推理阶段，向量层反向返回匹配的文档 ID 与相似度打分结果，支撑上层检索决策。

其中，L2层级的核心创新设计体现在存储架构的高适配性改造。我们对 VectorStore 做了完整的接口抽象，底层挂载两套接口完全兼容的后端引擎，双后端切换仅需修改配置字符串，无需改动核心业务代码，大幅提升架构通用性与迭代效率。同时我们采用存储分离架构，原始文本文档统一由独立 DocStore 持久化保管，不存入向量数据库，实现向量索引与原始文档的解耦存储，规避数据冗余与信息丢失问题。

本层架构的核心设计亮点，是**实现向量索引与原始文本的存储权责解耦**。我们对底层存储能力进行了清晰拆分：向量引擎仅承担索引计算职责，负责完成文档 ID 到向量特征的映射构建；完整的原始文本内容，则统一交由独立的 DocStore 模块进行持久化管控。

这一设计是保障双后端架构对等可切换的关键前提。如果将原始文本一并托管至向量库存储，麒麟官方引擎可通过 Meta 动态字段原生适配该能力，但轻量化 Numpy 后端并不支持对应逻辑。为实现两端对齐，就必须额外开发冗余的适配结构，最终导致两套后端的底层运行机制、功能权责无法统一，所谓的引擎可切换特性也会沦为形式化能力，丧失实际工程价值。

依托存储分离的架构方案，两套向量后端达成了完全的逻辑对等与功能归一。在实际工程部署中，无需改动任何核心业务代码，仅通过简单的参数配置即可完成后端切换，自由选用麒麟官方引擎或 Numpy 轻量后端，真正落地了低侵入、高灵活的可插拔架构设计。

2. 多源数据整合（对应赛题要求 (1)）

数据层接入三类异构数据、四个来源，产出 9,807 条真实、脱敏、去重后的标准化记录。

表3.1 数据来源与类型覆盖

数据类型	来源	说明
工具执行结果	Claude Code 会话日志、OpenClaw lucy agent 会话	主体数据源
用户行为数据	用户输入历史	偏好提取的情境侧输入
手动配置信息	手动配置文件	赛题点名的第三类

OpenClaw 侧的数据范围限定为 lucy 一个 agent。其余活跃 agent 绑定真实第三方消息渠道（QQ 群、飞书群），涉及第三方隐私，明确排除在采集范围之外；范围依据 openclaw.json 的 bindings 字段逐一核定。

质检与脱敏机制包含字段级 schema 校验、内容级去重、编码损坏检测、执行状态三态分类（成功/技术失败/用户拒绝）与递归脱敏。脱敏模块的设计经过两次真实凭据泄漏的修正：一次是 11 位 sudo 口令，长度短、不像随机串，首版规则未能覆盖；另一次是同一口令的大小写与后缀变体。规则最终固化为"结构化命令模式匹配 + 已知词根正则（不区分大小写）"两层，并要求递归处理任意深度的嵌套字段。另一个方向的风险是误伤：session_id 等 UUID 字段容易被"32 位以上随机串"规则打中，因此脱敏只作用于 tool_input / tool_output 的具体内容字段，不对整条记录一并处理。

赛题要求 (5) 的前半（敏感信息识别与过滤）到这里闭合，依据不是实现本身，是一份独立评测。此前全项目关于脱敏的测试只有 7 条手工用例，且只覆盖个别正则；"人工看过几十条"不能构成证据，召回率

与精确率才是。measure_anonymize_quality.py 用一份 46 条的标注测试集（27 条真含敏感内容，19 条形似而实为无害）对 _scrub_string() 逐条判定：

```
召回率   27/27 = 1.00        真正的敏感内容全部命中
精确率   27/27 = 1.00        打了码的没有一条其实不该打
误伤率   0/19  = 0.00        形似敏感的无害内容一条都没被误伤
```

19 条负例是专门冲着“误伤”这个方向造的：session_id 这类 UUID、长十六进制哈希，以及正文里谈到 API key 概念却不含真实值的句子。这 19 条全部未被误打码。

执行状态的三态分类纠正了一处实质偏差。原先的 is_error 二值统计掩盖了一件事：大量"出错"记录其实是用户拒绝、根本没执行。实测 5,174 条"出错"记录中，权限或审批拒绝占 91.3%，真实技术故障只占 2.4%。被拒绝的操作没有"怎么解决"这回事，本就不该进入历史案例库。

3. 知识记忆结构化整合（对应赛题要求 (3) 前半）

三类知识由三个挖掘器分别从事件关系中产出：

表3.2 三类知识的构成关系与产出

知识类型	由什么关系构成	当前产出
历史案例	失败 → 修复	31 条
工作流程知识	用户意图 → 工具调用序列 → 结果	54 条
可复用模板	跨会话反复出现的流程	1 条
合计		86 条

案例挖掘器经历了两轮方法修正。第一版启发式是"is_error=true 之后同会话 5 步内出现同工具的成功调用"，估算出 461 对；人工抽查后发现这个数字其中并不准确。大量来自上文那 91.3% 的权限拒绝记录。修正后的过滤分三步：(1) 从错误输出的文本内容判断是否真故障，不只看 is_error 标记；(2) 修复判定看后续调用输出是否不再含同类错误信号；(3) 相似度过滤，真正的修复与失败命令共享大量文本（同一主机、同一路径、同一段脚本），用最长公共子串占比衡量，低于 0.4 判定为"转去做别的事了"。第三步是在人工抽样 25 条候选、发现约 60% 仍是无关下一步操作之后才新增的。

冷启动可行性单独做过验证：把数据集按工具调用记录的真实起点切成"用了 1/2/3 周"的子集重跑挖掘器，1 周（987 次工具调用）产出 62 条真实知识单元，而不是 0。新用户没有历史记录时，记忆从空开始，随真实使用增量长起来，不需要造数据。

4. 偏好记忆动态捕捉（对应赛题要求 (2)）

本方案给"用户偏好"的操作性定义是：该用户在 Agent 交互轨迹中稳定表现出的操作模式与约束表达，没有区分具体动作由用户直接指定还是由 Agent 在用户授权下选择。依据有两条：赛题背景原文把"工具调用执行结果"列为记忆的核心数据源之一；下游价值在于"下次遇到同类情境能否复用既有模式"，与动作由谁发出无关。

提取机制是 L1 确定性规则，无需调用 LLM。操作习惯类以 intent 类别为情境、命令首动词为选择；VERB_INTENT_MAP 的约 60 组映射

来自真实数据中 995 个不同命令首动词的手工归类，不是随便列举的。三道门槛缺一不可：

```
min_trigger  = 8      情境出现太少，比例没有意义（4/4=100% 说明不了什么）
min_ratio    = 0.65   低于此值说明该情境下并无稳定倾向，是随机的
min_sessions = 2      只在一个会话里出现的是一次性做法，不是习惯
```

评测口径是本模块最重要的设计。用前期数据提取出的偏好去预测留出期的真实行为，命中率就是准确率；标准答案是留出期的真实行为记录，零人工标注、零循环论证。切分函数 split_by_time_keeping_sessions() 按会话起始时间排序，整个会话归入同一侧。这道防线针对的风险真实存在：本数据集中 Claude Code 会话可以跨天恢复，最长的一个横跨 23 天，纯按时间戳切分会把它拦腰切断，前半段预测自己的后半段就是切分泄漏。实测朴素时间戳切分确实切断了 3 个会话。

偏好的版本化没有重新发明机制。用户中途改变规则，也就是"旧偏好被新偏好推翻"，本质上就是知识冲突里的 T2 方法推翻，因此直接复用冲突模块已建成、已用 19 条真实冲突验证过的 adjudicate() 裁决器。

5. 新旧知识冲突处理（对应赛题要求 (3) 后半）

核心策略是给旧知识标注适用边界：检索默认返回新的，旧的保留且可追溯，任何一条都不会被新知识覆盖掉。

表3.3 冲突类型学（全部取自真实数据）

类型	定义	裁决规则
T1 事实过期	同一事实槽位，值变了	新值生效，旧值标注 as_of
T2 方法推翻	同一问题，后者否定前者的解法	新法生效，旧法保留适用条件
T3 环境变更批量失效	环境标识变了，一批知识同时过期	该环境下的知识批量标记待重验
T4 局部修订	同一知识文件某段被改写，不含强烈否定	段落粒度，新的生效

判定分三层，最关键的是三层之间的代码路径隔离：

L1 结构层（mine_conflicts.mine()，确定性，受变异测试保护）

同签名 + 跨会话/跨天 + diff 非空 → 候选池（19 条正例 + 128 条负例）

↓

L2 语义层（通读候选池做排序与解释，不产出 ground truth）

↓

L3 人工裁决（review_and_freeze_conflicts.py 的 ADJUDICATIONS，冻结为 ground truth）

ground truth 不能由规则或 LLM 自产，原因是被测对象正是 adjudicate() 这段确定性规则代码。冻结评测集的 36 对裁决不调用 adjudicate()，是逐条通读原始证据后手写的独立判断。此处存在一条边界：这 36 条裁决由人工逐一审查。它与"被测系统自评"仍属不同类别的风险，因为每条判断都能按 stable_key 回溯到原始记录，任何人可以重新核对。

结构层判据修正过两轮。第一轮，初版判据"新旧两侧 session_id 相同即不算冲突"会把跨天恢复的长驻会话整体误杀，改为"同会话且同一天"后，结构候选由 4 条增至 19 条（+375%）。第二轮，初版的"纯追加"判据 new.startswith(old) 只挡得住"接着往后写"一种模式，而索引类文件常见的是"在已有内容前面插入新记录"；改为"旧文本整体是新文本的子串"后，剔除了两条假阳性。

6. 关联检索优化与端侧存储（对应赛题要求 (4)）

本研究在基础朴素向量检索的基础上，搭建了四维度策略消融体系，依次验证**知识类型元数据先验、MMR 多样性重排、BM25字面混合检索、知识推翻单元降权**四类优化策略的独立增益，完成系统性的策略迭代与效果验证。

其中，知识元数据先验策略被拆解为两套对照方案，分别用于效果上界对标与工程落地部署。meta_oracle 方案依托标准答案的真实知识类型做精准过滤，仅用于测算理论性能上限，不具备实际部署价值。而 meta_prior 为可落地的轻量化先验机制：无需模型训练、无需依赖人工标签，仅通过统计同类知识文档的向量均值，生成各知识类型的质心向量；通过计算查询向量与质心向量的余弦相似度构建软先验权重，最终得分由向量检索得分与先验系数加权融合得到，计算公式为 final = vscore × (1 + β · prior)。整套机制无数据泄露风险，可直接适配端侧真实部署场景，且全程不做硬性过滤，保留了检索结果的完整性与灵活性。

针对 BM25混合检索链路，本研究修复了原有实现的核心缺陷。旧版本采用单字分词方式，且仅对 BM25得分做归一化处理，导致向量得分与文本得分尺度不统一、融合逻辑错位。优化后统一采用字符 bigram 分词口径，同时对向量检索、BM25文本检索两路得分，在候选集内分别完成 min-max 归一化，最终通过 hybrid_alpha 超参实现双路分数的精准加权融合，彻底解决尺度失配问题。

为保证超参取值稳定、实验结果可复现，本次所有核心超参 hybrid_alpha 与 prior_beta 均通过固定抽样子集完成定值。抽样规则

为在全部227道试题中，按读入顺序以固定步长间隔抽样，依次选取下标0、8、16……224的试题，最终得到29道固定子集。整套抽样流程无需随机种子，可完全复现。参数取值确定后全程冻结，不再迭代调整，规避了超参过拟合与实验随机性问题，保障实验结论严谨可靠。

7. 自然语言指令驱动的精准遗忘（对应赛题要求 (5) 后半）

赛题要求 (5) 的前半段（敏感信息识别与过滤）已经在 3.2 节实现并且有独立评测；本节处理后半段，用一句自然语言指令，从记忆里精确地移除某一条知识单元，且不伤及其余。

设计之前先确认了三个已经有的事实：向量存储的 `drop()` 是整集合删除，没有按条删除的接口；记忆服务以只读方式使用索引，索引的建立与更新归 `build_knowledge_index.py` 独立管理；已有的知识取代机制（supersede）本身也不删向量，而是读一张边表、在查询时把被取代的单元滤掉。这三条决定了遗忘的正确形态不是删除，而是 supersede 的同类——一张遗忘账本加一个查询时过滤器。

遗忘账本 `forget_ledger.jsonl` 只追加、不改写既有行，每条记录含时间戳、操作（`forget` / `unforget`）、被遗忘单元的 `stable_key` 与 `doc_id`、触发该操作的原始指令、单元正文的 SHA-256 以及确认标志。某个单元的生效状态取该 `stable_key` 下最后一条记录的操作。账本路径由启动参数 `--forget-ledger` 指定，评测使用独立账本，没有污染生产账本。过滤器与 `apply_supersede_filter` 同形：检索链取出候选之后、按 k 截断之前，按 `stable_key` 查账本生效状态，滤掉已经遗忘的单元。过滤必须发生在截断之前，否则遗忘掉 top-1 之后只会返回 k−1 条，这样的话行为就不一致了。

"精准"落在两处。第一是定位单位：以 stable_key 而非 doc_id 标识单元，前者由单元内容机算、重挖后不变，后者是索引位置、重建即漂移。第二是两步确认：自然语言天然有歧义，接口先回显候选、由调用方确认后才写账本，避免忘错。表3.4 列出接口契约。

表3.4 遗忘账本接口契约

接口	请求	行为
POST /forget（第一步）	{"instruction": "…", "k": 5}	把指令当作查询走现有检索链，返回候选列表（doc_id、stable_key、类型、正文前 200 字），标注"未执行遗忘"
POST /forget（第二步）	{"instruction": "…", "confirm_doc_id": N}	校验 N 在第一步候选内，追加账本，返回被遗忘单元与账本长度
POST /unforget	{"stable_key": "…"}	追加一条反向记录，撤销遗忘
GET /forgotten	—	列出当前生效的全部遗忘

定位一步直接复用 _retrieve_ablation()，没有另写指令解析器：指令怎么描述一条记忆，检索就怎么找到它，这是"自然语言指令驱动"最直接的实现方式，也意味着定位准确率的上限就是检索本身的语义能力。

这个设计的代价与收益都很明确。收益是冻结的 17 个文件、语料与索引一字节不动；遗忘可逆（撤销即追加一条反向记录）；可审计（谁、何时、凭什么指令、忘了哪条，账本里全都有）；与现有架构一

致，改动面小。代价是被遗忘的向量仍在索引里，只是查不到——若需求是物理清除，需要重建索引，这属于索引管理而非记忆服务的职责。

四、评测与结果

1. 实验环境

表4.1 执行环境

项	配置
目标机	银河麒麟桌面操作系统 V11（磐石架构），2 vCPU / 3.9 GB RAM
向量化	KYSDK Embedding，768 维，官方 `kylin-ai-runtime` 常驻
向量引擎	kylin-ai-vector-engine 1.2.0.1（Milvus Lite 麒麟分支），Unix socket + gRPC
对照后端	numpy 归一化矩阵 + BLAS

麒麟 V11 采用不可变系统设计，/usr 覆盖层在重启后必定退回出厂状态（经跳闸断电与正常重启两种情况验证，PVE 快照亦无法恢复，因为这些改动从未落盘）。本项目据此建立了一键环境恢复脚本 restore_env.sh，存放于持久分区 /home，并配置了开机自动恢复：系统层由 root 的 @reboot crontab 触发（/var/spool/cron/crontabs 位于独立 ext4 分区，持久性确定），图形会话层由桌面 autostart 触发（kylin-ai-runtime 必须在图形会话下才能拉起）。该机制已经得到验证，768 维结果三次独立复现、数值逐位一致。

2. 评测方法学

在没有外部标准答案的条件下，本项目对评测方法本身施加了四条约束：

1. 冻结：评测集连同其 sha256 与语料哈希一并写入 *_frozen.json，写文件显式指定 newline="\n" 以避免 CRLF 导致跨平台哈希不一致；版本号必须随内容变，分档统计之和必须等于总数，两道防呆断言。
2. 同时上报两个指标：不仅给出准确率的单点估算值，还要附带 Wilson 95%置信区间，不能只用单个准确率数字直接下结论
3. 每个检索指标同时给出纯词面基线（不含任何 embedding，仅字符 bigram + TF-IDF 余弦）。
4. 不合成总体数字：类别、难度分档的证据强度与样本量不同，合成会互相掩盖。

评测方法本身也经过一轮调整。早期版本我们用“简单题目 R@1是难题1.97倍”来证明模型具备语义理解能力，复核的时候发现这个论证站不住脚。纯文字匹配基线，在简单/难题之间本身差距就很大；简单题的规则要求保留关键词，所以单纯关键词匹配，也能拉出这个倍数，没法区分模型是真理解语义，还是只会匹配文字。修改之后，我们改用难题场景下，模型相对基线的提升幅度，作为判断语义理解能力的证据。

3. 知识检索召回率（硬指标：≥85%）

评测集 v1.4.0，227 题，三档难度：close 74 题 / paraphrase 77 题 / hard 76 题。语料 86 条知识单元。

表4.2 总体召回率（模型 vs 纯词面基线）

K	模型（官方引擎）	纯词面基线	点估计 ≥85%？	95% CI 下界
recall@1	52.4%	35.2%	否	45.9%
recall@3	76.2%	46.7%	否	70.3%
recall@5	82.4%	52.4%	否	76.9%
recall@10	86.3%	57.3%	是	81.3%（否）

表4.3 按难度分档（模型 / 基线）

难度	n	R@1	R@3	R@5	R@10
close 模型	74	70.3%	89.2%	93.2%	94.6%
close 基线	74	59.5%	73.0%	82.4%	89.2%
paraphrase 模型	77	54.5%	77.9%	84.4%	89.6%
paraphrase 基线	77	44.2%	58.4%	64.9%	71.4%
hard 模型	76	32.9%	61.8%	69.7%	75.0%
hard 基线	76	2.6%	9.2%	10.5%	11.8%

主指标取 hard 档 R@1：模型 32.9%，纯词面基线 2.6%，领先 30.3 个百分点。 取这个指标而不取总体数字，理由是可证伪性。close 档题目按生成规则要求保留知识单元中的关键词，等于从被检索文档里抠关键词造出来的；实测纯词面方法在该档直接反超模型（BM25 89.0%，模型 85.4%，v1.3.0 数据），用 close 档论证语义理解属于循环论证。hard 档题目明确要求不出现原文的工具名、命令名与专有名词，纯词面方法天然效果不理想，实测基线只有 2.6%，模型仍能命中三分之一。

以上结果附三条限定。

1. 就算是模型自身，难题 R@1对比上一版也从44.8%下降到32.9%。领先幅度变大的原因，是新版难题更难、基线跌得更狠，并不是模型本身变强了。
2. 总体 recall@10单点数值86.3%，超过门槛，但95%置信区间下限只有81.3%，达不到85%的标准。报告标记这个指标未达标，claims.json 中的 C-255标记保留红色。如果单纯调低门槛、只看点估计值判定，属于简化检查，结论不够严谨。
3. 相对 v1.3.0（169 题 / 95 语料，recall@10 = 91.7%）的下降来自评测集变难，新增了整整一档 paraphrase 77 题，检索质量本身没有下降；按子采样曲线，语料由 95 降至 86 本应让召回率略升 1.5~2.1 pp。跨版本数字不可以直接比较。

两个后端的结果逐位一致：kylin 与 numpy 四档全部是 52.4 / 76.2 / 82.4 / 86.3%。同一套 embedding 加暴力余弦排序，后端只影响延迟、不影响排序，这构成一次隐含的交叉验证。2026-08-15 这项一致性一度不成立（numpy 后端 R@1 只有 2.6%），根因是整链重跑时只重建了官方引擎的 collection、漏了 numpy 那份，doc_id 全部错位。这条错误没有被任何检查器拦下，是肉眼看两行数字对不上才发现的，记录保留在《勘误与修正记录》中。

4. 关联检索消融

表4.4 七配置消融对照（语料 86 条）

配置	R@1	R@5	R@10	R@10 95% CI	与 naive 排序差异题数
naive	52.4	82.4	86.3	[81.3, 90.2]	—
meta_oracle（上界，非系统召回率）	55.5	85.9	91.2	[86.8, 94.2]	36
meta_prior（真实策略，零泄漏）	52.0	82.8	87.2	[82.3, 91.0]	14
mmr	52.4	71.4	78.4	[72.6, 83.3]	58
bm25（修复后，α=0.5）	45.8	78.0	86.3	[81.3, 90.2]	109
supersede（预期内 no-op）	52.4	82.4	86.3	[81.3, 90.2]	0
all（meta_prior+mmr+bm25+supersede）	46.3	76.2	86.8	[81.8, 90.6]	110

四条轴的结果分别如下。mmr 下降 7.9 pp：本评测是单 gold 任务，多样性重排本来就该掉，不需要修。supersede 为 0，是已知的 no-op，代码 docstring 里已经声明。bm25 在分词与归一化缺陷修复后由 69.6% 回升到 86.3%，与 naive 打平。meta 拆成上界 91.2% 与零泄漏真实策略 87.2% 两档，后者拿到了 oracle 约 18% 的空间。四条轴综合后，all 档相对 naive 是 197/227 vs 196/227，净 +1 题（+0.44 pp，size_effect = 0.00 pp，未移除任何 superseded 单元）。

配对检验说明：7组方案使用完全相同的227道题目，属于配对实验，所以选用 McNemar 检验，而不是独立样本区间检验。独立区间检验对配对样本过于保守，属于方法误用。结果：

表4.5 七配置相对 naive 的 McNemar 配对检验（227 题）

配置 vs naive	净题数	McNemar p	结论
meta_oracle（上界）	+11	0.0010	显著，但是上界不是系统能力
meta_prior	+2	0.50	不显著
bm25（修复后）	±0（b=c=8）	1.00	精确中性：8 题得、8 题失
mmr	−18	0.0014	显著变差
all	+1	1.00	不显著

结论：修正 BM25 中文分词（单字→字符 bigram）与双路分数归一化后，混合检索由显著有害变为中性；在 86 条语料规模上，四条关联检索轴均没有观察到统计上可辨的正向收益。 mmr 的 −18 题（$p<0.01$）是一条方向为负但确定的结论：单 gold 检索任务上多样性重排被证伪。

all 档包含了已在 $p<0.01$ 水平证明有害的 mmr，故其 86.8% 低于 meta_prior 单档的 87.2%。实际部署应排除 mmr——这个判断的依据是 mmr 自身的显著性，不是“排除后数字更好看”，两者性质不同。

与修复前对比：修复前 all 档为 79.3%，比 naive 低 7.0 pp；修复后 86.8%，首次超过 naive。修复涉及三处缺陷。一是 meta 档消费了题目的 gold 类型标签，现已收紧为只有 meta_oracle 一处可消费，并新增第 19 项泄漏检测器，对所有非 oracle 配置断言"换 gold_type 结果必须一致

"，同时做阳性对照证明检测器本身有辨别力。二是 BM25 的单字分词与尺度失配。三是缺少可部署的元数据策略，为此新增了 meta_prior。

5. 偏好提取准确率（硬指标：≥85%）

表4.6 留出期行为预测准确率（三类分开报，不合成）

类别	n（留出期出现次数）	提取器准确率	全局众数基线	提升幅度	提升区间（保守）
操作习惯	1352	90.0% [88.3, 91.5]	20.5% [18.4, 22.7]	+69.5 pp	[65.6, 73.1]
输出风格	44	68.2% [53.4, 80.0]	68.2%（恒等）	0.0 pp	[−26.6, 26.6]（跨 0）
安全策略	0	—	—	—	—

三类里只有操作习惯类在统计意义上确信达标：准确率区间下界 88.3% 高于 85% 门槛，提升区间下界 65.6% 大于 0。

输出风格类没有提升，属于结构性问题，和提取器本身的好坏没有关系。这个类别目前只有 doc_format 这一种触发场景，类别基线和这个单一场景下的多数结果数学上相等。后续优化思路，拓宽输出风格的场景细分，比如按照文档目录、项目上下文做区分。

安全策略类实测是空结果。数据集中 user_rejected 记录共 865 条、占 9.34%，其中 798 条（92.3%）来自同一天同一会话，与案例挖掘器早已记录的"单日批处理污染"是同一个问题，这次出现在 execution_status 字段上。排除异常日之后，正常日的拒绝率只有 0.8%，

没有任何命令类型的拒绝率能得到统计支持：rm 这类高风险操作的拒绝率反而只有 3.6%，ip、df 等只读命令的高拒绝率全部是异常日产物。

修正后的第一版实现仍产出 52 条"安全策略偏好"，逐条核查发现全部是"对 X 的默认处置是 allowed，ratio ≈ 0.97~1.0"，也就是在复读全局低拒绝率基线，够不上策略。最终改法是只统计"拒绝"为多数的组，"允许是多数"的组一律不产出：它们的支持度并不低，问题在于本就不该被称为策略。该类按空结果报告。

阴性验证的设计假设同样被实测推翻。原设想的阴性验证是"打乱时间顺序后准确率必须掉到基线水平"，实测不成立：随机打乱后操作习惯准确率是 94.6%，比按时间切分的 90.0% 还高。这符合"习惯"的本意，真正的习惯本来就该跨时间稳定；如果打乱后暴跌，反倒说明提取到的是短期上下文效应。该项已改为直接验证真正要防的风险，即会话跨侧泄漏必须降到零。

版本化实测检出 1 条真实的偏好版本变更：disk_usage 情境下 du → df，裁决为 T2 新推翻旧。该条的解读存在余地：du 和 df 问的是不同问题，这条"变更"也可能说明 disk_usage 这个 intent 类别粒度偏粗，把两种用途合并了。该条按原样记录。

6. 知识冲突处理正确率（硬指标：≥88%）

评测集分两部分，规则冻结先于留出集数据产生：

- 开发集 36 对（19 正 + 17 负），冻结版本 v1.3.0 —— 规则在其上开发

- 留出集 60 对，来自 D1–D7 采集（2026-08-26 起）—— 判定规则于 2026-08-27 冻结

（mine_conflicts.py 哈希 bebd5ccd），冻结时这批数据一条都还不存在

表4.7 判定准确率（三个口径分开报）

口径	结果	Wilson 95% CI	判定
开发集 n=36（规则在其上开发）	35/36 = 97.2%	[85.8%, 99.5%]	下界不过
留出集 n=60（规则从未见过）	56/60 = 93.3%	[84.1%, 97.4%]	下界不过
合并 n=96（全部标注数据）	91/96 = 94.8%	[88.4%, 97.8%]	达标

合并口径达标，原因是样本总量增加，不是泛化效果比开发集更好。点估计94.8%落在开发集97.2%、留出集93.3%中间。留出集单独置信区间下限84.1%，没有达到门槛。两句话需要一起看：合并数据集包含开发阶段见过的36条样本。

从这两点说明：开发阶段两次结构修改（same_sitting 口径、append_only 匹配方式）属于 bug 修复，不是专门调优精度；留出集、开发集的点估计差距落在各自置信区间内，没有明显性能退化

表4.8 留出集分层准确率

层	命中	准确率	Wilson 95% CI
C 语义更正（需读懂内容才能判）	15/15	100.0%	[79.6%, 100.0%]
B 环境失效（重启导致前提消失）	8/8	100.0%	[67.6%, 100.0%]
A 数值漂移（仅读数变化）	5/7	71.4%	[35.9%, 91.8%]
负例	28/30	93.3%	[78.7%, 98.2%]

分层是按照机器可识别的规则划分，抽样数量（C 层15 / B 层8 / A 层7 / 负例30）和随机种子，在看到分层结果之前，就写进协议并且提交，参考文档《留出集抽样与裁决-协议 v1.0》。C 层15/15是权重最高的一组，这一组必须读懂文本内容，才能做出正确判断。

表4.9 裁决三层（n=19，v1.3.0 口径）

子指标	结果	Wilson 95% CI
谁生效 verdict	19/19 = 100.0%	[83.2%, 100.0%]
类型 T1~T4	8/19 = 42.1%	[23.1%, 63.7%]
reason_kind	17/19 = 89.5%	[68.6%, 97.1%]
回溯完整性 retrieve_old()	34/34 全对	二元硬断言

reason_kind 由 v1.2.0 时的 94.4%（17/18）降为 89.5%（17/19）：新增的 K-0052 被规则判错了 reason_kind（人工判“旧结论当时为假”，规则判“前提消失”）。该数字往不利方向变动且如实记录， 是评测集修订未被结果导向的直接证据。

6.1 标注方法与可靠性

判定依据《冲突判定标注规范》v1.0.2 —— 该定义此前从未写成文字，只隐含在结构过滤代码里， 2026-08-28 补写并冻结。标注采用盲测：打乱顺序，只显示新旧两段原文， 隐藏结构标签、来源与时间戳（时间戳会泄露"是否跨天"，正是要防的结构暗示）。

人工全量标注 73 条 + 5 条重复题（同一条隔 29~70 题出现两次）：

重复题前后一致　　5 / 5 = 100%　　← 人工标注唯一能自证的可靠性指标

逐条写了裁定理由　78 / 78

6.2 留出集 4 处误判：单一根因，已诊断

四处全部源于一个交互缺陷，不是任何一边单独的错：

提示词要求笔记写成　　「读数：<值>（原为 <旧值>，已不再是）」

挖掘器 append_only 用　「旧文本 ⊂ 新文本」子串匹配

两者交互 → 双向失效

- 史注漂移（A 层 2 条）：当前值未变、只有史注变 → 误判为冲突
- 假追加（负例 2 条）：旧值被写进史注 → 旧文本成为子串 → 误判为纯追加

该案例已经补充进标注规范 v1.0.2。代码没有修改、指标没有重算，冻结状态保持不变；如果两处逻辑全部修复，留出集结果理论上可以 60/60。

6.3 样本量的历史限制（已解除）

n=36 时区间过宽曾是主要制约，来源有二：S3（案例签名多解法）矿脉实测为空； S2 的长驻会话与纯追加两个假阳性来源占掉了 148 条编辑事件中的 128 条。 D1–D7 采集产出 509 条候选（235 条结构正例）后该限制解除，n 由 36 增至 96。

“谁生效”指标19/19=100%，由问题本身结构决定。verdict 规则本质是：永远采信时间更晚的断言。本数据集19条真实案例里，时间靠后的断言全部正确。这个特性来自真实使用场景，人不会无故撤销已经证伪的旧结论。规则和数据集这一特性匹配；比时序判断更难的语义判断，还没有验证。

类型判断准确率44.4%，是三层指标里最低的一项。加入环境变更关键词前为38.9%，加入之后到44.4%。原因是 T1与 T3边界，在“事实随环境更新”这类场景本身就模糊。同一个物理事件，从“这套架构描述还成立吗”看属于 T3；从“这块盘现在是什么状态”看属于 T1，人工标注本身也会出现分歧。n=18样本太少，不适合继续调参，该项不再优化。

S3 矿脉的实测结果为空。设计方案原估算该矿脉可产出 5~8 组真冲突；实测按签名分组分出 4 组、5 对跨会话候选，逐对核查源头后全部为假阳性：签名截断窗口落在"Traceback (most recent call last)"这类所有 Python 报错共享的样板文本里，没有区分度。追加一道失败命令本身的文本重叠度过滤后（阈值 0.3，4 组假阳性的实测重叠度全部落在

0.04~0.20），S3 正例为 0 条。当前阈值未下调，该结果表明这批数据中 case 类知识尚未形成该矿脉。

7. 检索响应延迟与端侧轻量化（硬指标：≤500 ms）

表4.10 向量化耗时分档实测（P95）

文本长度	10 字	30 字	128 字	512 字	2048 字
P95	36 ms	68.4 ms	240 ms	824 ms	4170 ms

立项的时候，我们假设每次调用 embedding 接口，都需要重新加载 339MB 模型，常驻服务架构也基于这个假设设计。实测推翻这个猜想，新旧架构 P50分别486.7ms、484.2ms，提速几乎没有。因为模型常驻在 kylin-ai-runtime 守护进程内，客户端 text_embedding_init_model() 只做一次握手。

真正瓶颈是文本长度，耗时和字数近似成正比，吞吐大约936字/秒。查询文本和文档文本要分开统计：查询一般是短文本，30字级别；文档文本只在离线建索引的时候一次性计算。混在一起统计，会得出 P95 3348ms 这种错误结论，本项目第一轮就踩过这个坑。

表4.11 双后端量化对比（20,000 条 × 768 维，top-10）

指标	官方麒麟向量引擎	轻量 numpy	倍数关系
冷启动	58.5 ms	0.2 ms	numpy 快 279×
建索引（2 万条）	14.58 s	1.66 s	numpy 快 8.8×
检索 P50	14.48 ms	2.89 ms	numpy 快 5.0×
检索 P95	15.84 ms	6.60 ms	见下注
常驻内存	141.6 MB	59.3 MB	numpy 省 2.39×
磁盘占用	≈ 58.6 MB	58.7 MB	大致相当

内存构成：官方引擎141.6MB = 守护进程增量114.5MB + 桥接进程27.1MB；numpy 方案59.3MB 几乎就是原始向量数据（20000 ×768 ×4B =58.6MB），几乎没有额外开销。
numpy 的 P95（6.60ms）不能直接和官方引擎对比，n=20样本刚好命中 BLAS 线程池预热。对比倍数优先看 P50。

引擎侧磁盘读数曾经0.0MB 是测量假象：default.db 文件已经膨胀，SQLite 删除数据不会自动释放空间，本次写入复用空闲块。

端到端延迟，在86条真实语料、真实查询上逐条统计，取分位数：kylin 方案 P95 111.70ms / numpy 方案 P95 118.32ms，只占用500ms 预算的22.3% / 23.7%，还有76%以上余量，这项硬指标达标。延迟大头约85ms，占76%~85%，来自赛题强制调用的官方 embedding SDK。检索后端选择，只会带来个位数到十几毫秒差别，对比500ms 预算，不会造成本质影响

立项时推测"官方引擎作为专业向量库，在轻量化上可能优于自建方案"，实测结果相反：numpy 后端在冷启动、建索引、检索延迟、常驻内存四个维度全面占优，内存只有官方引擎的 40%。这符合工程直觉。2 万条 × 768 维只有 6100 万次乘加，BLAS 一次矩阵乘法就算完了；官方引擎则要走 gRPC 序列化、跨进程通信与索引维护，这些固定开销在该数据量级下反而成了主要成本，专业向量库的优势要到千万级数据量才显现。

最终仍默认选用官方麒麟向量引擎，理由是端侧适配完整性与可扩展上限：引擎配置 max_count: 400000，而 numpy 方案内存随数据量线

性增长，40 万条将占约 1.2 GB，在 3.9 GB 端侧设备上会很紧张。代价是 2.4 倍内存与约 12 ms 延迟。numpy 后端保留下来，作为内存受限场景的一键切换选项。

8. 硬指标汇总

表4.12 四项硬指标达标情况

硬指标	门槛	点估计	95% CI 下界	判定
偏好提取准确率	≥85%	90.0%（操作习惯类）	88.3%	达标（限该类别）
		68.2%（输出风格类）	53.4%	提升为 0，见 4.5 节
知识检索召回率	≥85%	86.3%（recall@10）	81.3%	点估计过线、区间下界不过
知识检索响应时间	≤500 ms	111.70 ms（P95）	—	达标，余量 77.7%
知识冲突处理正确率	≥88%	94.8%（合并 n=96）	88.4%	达标
		93.3%（留出集 n=60 单独）	84.1%	单独口径不过，见 4.6 节

没有在统计意义上达标的指标，按照赛题原文要求，写清楚优化方向：

召回率：单纯扩充语料解决不了这个问题，这个结论可以量化。按照本项目实测语料规模效应（每增加50条语料，recall@10大约下降1.2个百分点），结合文档聚类的事实（227题覆盖77个 gold 文档，平均2.9题/文档），语料和题目同比例扩大后的结果如下表：

表4.13 语料与题目同比例扩大后的 recall@10 外推

语料	题数	recall@10	CI 下界
86（现状）	249	86.3%	81.5%
142	412	85.0%	81.2%
200	580	83.6%	80.4%
500	1450	76.4%	74.2%

下界从不上升。 点估计随语料线性下降，而置信区间只按 n^(−1/2) 收窄——线性快于根号， 因此从第一条新增文档起下界即单调走低，任何规模都到不了 85%。 唯一有效的杠杆是压低语料规模效应的斜率，即让检索在语料变大时不掉分； 而 4.4 节 的消融表明四条关联检索轴在当前规模下均无统计上可辨的正向收益。 该链条本身即为本指标的优化方向：先证明关联检索在更大语料上有效，再谈扩充语料。

该曲线由 86 条向下子采样测得，向上外推属外推而非测量；真实系统在更大规模上可能走平。 但要据此翻盘需先把语料加上去才能验证，本届时间窗内不可行。

偏好的输出风格类：拓宽情境切分粒度（当前仅 doc_format 单一触发情境， 类别级基线与该情境众数在数学上恒等）。

冲突处理：该项已于 2026-09-07 由留出集裁决补足样本量（n 36→96）并达标； 留出集单独口径仍不过线，优化方向见 4.6 节 诊断出的 append_only 子串判据与史注格式的交互缺陷。

9. 模型消融 Benchmark：记忆机制是否依赖生成模型规模

下面两节做两组正交对照实验，用来支撑"方案效果来自机制本身设计"这个论点，不属于赛题四项硬性指标。本节更换生成大模型；下一节更换向量编码模型，其余所有环节固定不变。

固定检索管线、评测数据集、prompt，只替换生成模型。4档模型 ×2 组条件（不调用记忆接口 / 调用记忆接口），一共8组，13道任务，全部跑完。
判分使用零 LLM 裁判，严格字符串匹配口径。三种可能结果的解释，在跑实验之前，就写死在 deltas_prereg_v2.md，时间戳早于所有结果文件。

"不调用记忆接口"这一组，是真的不请求这个 HTTP 接口，不是删掉提示词里的一段文本。两组唯一差别，就是记忆服务是否启用。

表4.14 4 档模型 × 2 条件（13 题，严格子串口径）

生成模型	不调记忆	调记忆	Δ
Qwen3.5-2B Q4_K_M	2/13	5/13	+3
Qwen3.5-4B Q4_K_M	3/13	7/13	+4
Qwen3.5-9B Q4_K_M	2/13	7/13	+5
Gemini-3.1-Pro maxthinking（云端，不满足端侧约束，仅作天花板参照）	3/13	8/13	+5

n=13 的样本量撑不起显著性推断，因此本节不做显著性结论，只报条数与方向。判分口径的选择还会 同时抬高一个结论、压低另一个结论：放宽为有序口径会让阶梯更陡，却把「9B 追平云端」从 7:8 拉开成 8:11。与其等人来问判据是不是调出来的，不如把三种口径下的稳健性检验一并给出， 并据此为每条结论标注强度。

表4.15 结论强度分级（据三种判分口径 + 留一法检验）

结论	强度	依据
记忆机制在四档模型上均产生正增益	稳健	三种口径 × 四档 = 12 格全为正（+3~+7）
增益随模型能力单调不减	口径依赖，且脆弱	宽口径下不成立；留一法剔 T-02 或 T-13 即破
9B 已追平前沿云端模型	口径依赖	有序口径下为 8 : 11，不成立
端侧最小可用规模约为 4B	方向性	2B 与 4B 调记忆后差 2 条；剔 T-07 后两档 Δ 并平

只有第一条可以作肯定陈述，其余三条必须连同口径条件一并给出。本报告照此办理。 记忆机制有效这一条不依赖判分口径的选择，“能力源自机制设计”的证据没有问题。 但“2B 够用”不成立：张力题上 2B 调记忆后仍以 1:4 落后于云端模型调记忆， 记忆机制的收益依赖模型有能力把取回的内容重组出来，2B 用不满。

10. BGE-M3 对照：赛题指定的向量化接口是否构成短板

官方答疑明确允许在对比实验中引入 BGE-M3 作对照组。本节固定同一份 86 条知识单元与 227 道题 （texts.jsonl 逐字节固定）、同一份检索代码，只换 embedding：赛题指定的 kylin-coreai-embedding（GTE-base 多语言，768 维）对照 BGE-M3 dense（1024 维）。 headline 取 naive 档（纯向量检索），以隔离 embedding 本身的差异。

表4.16 naive 档四档召回率对照（227 题）

指标	赛题指定 KYSDK Embedding	BGE-M3 (dense)	差（BGE − 指定）
R@1	52.42%（119/227）	50.66%（115/227）	−4 题
R@3	76.21%（173/227）	70.04%（159/227）	−14 题
R@5	82.38%（187/227）	77.09%（175/227）	−12 题
R@10	86.34%（196/227）	87.22%（198/227）	+2 题

结论是赛题指定的 embedding 接口在本任务上不构成短板。 R@10 两侧基本持平（差 2 题， McNemar 配对下只系统命中 14 题、只 BGE 命中 16 题，无明显方向）；R@3 与 R@5 指定接口反而 各多命中 12~14 题。这一侧结果有利，因此按预登记方案逐条自查了三个最容易把对照做坏的 地方：L2 归一化两侧无条件执行；只用 BGE-M3 的 dense 一路，不开 sparse 与 colbert （多向量混合等于拿集成方法比单编码器，那是另一个实验）；截断统计显示 86 条文档最长仅 393 token、 227 条查询最长 62 token，两侧均没有发生截断。检索代码本身用已知答案自检：retrieve.py 逐位 复现了 claims.json 的 C-265（R@1 = 0.5242）与 C-266（R@10 = 0.8634）。

本节的范围边界如实声明：只做了 naive 档，系统档（元数据软先验 / MMR / BM25）未做。 时间盒内优先隔离 embedding 本身的差异，而系统档的后处理会稀释 embedding 的贡献。 测量记录 M-211 与对应的

7 条 claims（C-410~416）已接进 可复现性体系/，数字已纳入机器守护；不依赖那套体系时，BGE-M3对照/复现.md 里 3 条纯 numpy 命令也能在任何机器上重跑本节全部数字。

11. 精准遗忘：定位、生效与副作用

本节是 3.7 节的评测。三个指标全部是集合比较，零 LLM 裁判，计分层纯 Python（L1），生成层需要 embedding 服务（L2），与模型消融、BGE-M3 对照同一套分层。

评测复用冻结评测集 `eval_set.jsonl`（227 题，覆盖 77 个 gold 文档）。它本身就是"某条记忆还能不能被检索到"的探针：遗忘单元 D 之前，gold 为 D 的题应能在 top-10 里命中 D；遗忘之后，这些题的 top-10 必须不再含 D（遗忘生效），而 gold 不为 D 的其余各题，top-10 集合必须与遗忘前逐题相同（无副作用）。任务集 `forget_tasks.jsonl` 共 20 条，case 与 workflow 各 10 条，覆盖 20 个不同的 gold 文档；`gold_stable_key` 由语料机算，不进行手填。指令措辞刻意不抄评测题原文，而是描述"这条记忆是关于什么的"，否则定位准确率会靠字面重合虚高。

流程对每条任务独立执行：清空评测账本；跑 227 题得到基线 top-10 集合（只跑一次，全部任务共用）；对每条任务，先以指令调用 `/forget` 第一步，记录 top-1 是否命中 gold（定位准确率）；再以 gold 强制确认——定位与遗忘是两个独立能力，分开测，定位错了在真实流程里本就会被确认步骤拦下；然后重跑 227 题，检查 gold 为 D 的题是否已不含 D（遗忘生效率）、其余题的集合是否逐题相同（副作用率）；

最后 /unforget 复位。所有逐题前后集合落盘于 runs_forget.jsonl，score_forget.py 从中重算三个指标。

三个指标的预期在跑评测之前写入 预登记.md（时间戳早于任何结果文件）：定位准确率 60%~85%，理由是措辞刻意不同于评测题，不该是 100%，低于 50% 则说明措辞写偏了，允许改指令措辞、不允许改判据；遗忘生效率 100%，因为过滤在截断之前，理论上必达，不达即 bug；副作用率 100%，因为过滤只按 stable_key 精确匹配滤掉一个单元，不达即 bug。任何一项不达预期，处理原则是先查实现，不调判据、不删任务。

表4.17 精准遗忘三指标（20 条任务，227 题探针）

指标	定义	实测	预登记预期	判定
定位准确率	第一步 top-1 doc_id 命中 gold 的任务占比	55.0%（11/20）	60%~85%	低于预期下限
遗忘生效率	确认后 gold 为 D 的题中，D 不在 top-10 的占比	100%（58/58）	100%	符合
副作用率（原始口径）	遗忘前后 top-10 集合逐题相同的占比，任何变化都计入	90.3%（4049/4482）	—	对照值，非结论
副作用率（真异常口径）	剔除"目标单元本就在该题 top-10 内"的正常回填后	100%（真异常 0/4482）	100%	符合

定位准确率是本节唯一没有达到预期的一项，过程我们如实交代了。首轮实测为 30%（6/20），低于预登记设定的 50% 下限；按预登记事先授权的路径，改写了 14 条未命中任务的指令措辞，重跑一次得到 55%，只改这一次，判据与任务数均未动。55% 仍低于预登记的 60% 下限，本报告按不达预期报告。这个数字的含义是：定位能力就是检索本身在"措辞刻意避开原文"条件下的语义能力，与 4.3 节 hard 档的表现属同一类边界；两步确认机制正是为这个边界设计的——定位不需要每次都对，错了会被确认步骤拦下，不会忘错。

副作用率有一个容易误判的边界，预登记里提前写明了判据。原始口径的 90.3% 里，433 条"遗忘前后集合不同"逐条核验后，全部都是同一种情况：被遗忘的单元本来就真实排在某道无关题的 top-10 里，遗忘它之后第 10 名由下一个候选回填。这是过滤器工作正常的表现，不是漏洞。真正的副作用应当是"目标单元不在该题原始 top-10 内，结果却变了"，这样的真异常实测为 0。两个口径我们都报了，原始值作对照，真异常口径作结论。

本节数字已接入可复现性体系：测量记录 M-213，主张 C-430 至 C-435 共 6 条，其中定位准确率那条的备注写明了首轮 30% 与改措辞一次的经过。`score_forget.py` 在任何机器上可从 `runs_forget.jsonl` 纯 Python 重算全部三个指标。

五、可复现性体系

自建评测集会遇到一个绕不开的问题：第三方没有办法重新拿到一模一样的数据。这个项目不打算搞很复杂的证据格式，我们的思路是保证可复现性和可证伪性。

这套系统没法保证项目没造假。日志、测量记录、哈希值都能伪造，AI 随手就能做出格式看起来没问题的文件。觉得有证据就等于数据可信，其实是自欺欺人。可信度靠的是方便别人验证，而不是做到绝对不能造假。

方法可复现，不等于原始数据可复现。数据提取脚本任何人在自己机器上跑一遍，都能生成格式相同的数据集，这就是方法可复现。但本项目这 9807 条记录，来自项目成员本机真实使用记录，别人没办法重新采集到完全一模一样的原始数据，也就是数据不可复现。所以，如果要核验依赖这份特定数据得出的指标，第三方只能用我们交付的这份数据集重新跑一遍，核对结果是否一致；想重新采集一份新数据拿来对比，这条路走不通。

体系按对环境的依赖分层：

表5.1 L1 / L2 分层

层	依赖	主张数	覆盖
L1	Python 3.8+ 与 numpy，任何机器	115	数据集统计、知识可挖掘量、脱敏规则、暴力检索对比、numpy 后端性能与内存、语义检索正确性、挖掘产出量、评测集冻结统计、知识检索召回率全量复算、偏好

			提取（M-401~404）、知识冲突判定/裁决/回溯 + 规则冻结锚点 + 留出集裁决（M-301~306）、BGE-M3 对照检索计分（M-211）、模型消融 4×2 计分与稳健性检验（M-212）
L2	银河麒麟 V11 + KYSDK	30	768 维双路验证、embedding 延迟分档、官方向量引擎资源与正确性、召回率的真机现场复算、关联检索消融阶段二（M-210）

总共有 145 条数字结论可以机器自动核对，其中 115 条不用银河麒麟环境。L1、L2 怎么划分、条目数量以《可复现性体系 / 当前数字看板.md》为准，这张表直接照搬。想让 L1 能核验语义检索是否正确，关键就是把官方 KYSDK 算出来的向量固定导出，一起放到交付包里。里面包含 86 条真实知识单元，还有冻结评测集 227 道题对应的全部向量。别人不用搭麒麟环境，就能重新算相似度，复现报告里一样的排序、分数和召回率。但有一点 L1 验证不了：没法证明这些向量确实是官方 KYSDK 跑出来的。这个属于 L2 层面的结论，要在麒麟机器上实测才行。两条验证路径合在一起才是完整证据链，L1 本身不负责核验这一点。

收尾的时候，我们把两组对照实验也加到这套核验体系里：BGE-M3 对照（M-211）和模型消融 4×2（M-212），总共多了 17 条结论。模型消融这一块的登记方式要单独讲下：claims.json 里面不光存数字，还要写清楚结论的靠谱程度。像 “四档 × 三口径都有正向增益”，这个稳定结论单独一条；“Δ 阶梯单调” 这个说法要看口径，必须搭配 “宽口径下不成立” 一起看；还有 “9B 追平云端”，要注明在有

序口径下这个结论不成立。哪怕是自带限制、甚至会削弱我们结论的条目，也全都放进机器校验里。所以这套东西，并不是只用来包装对我们有利的数据。

test_detectors.py 现在一共申报了 164 项检测，涵盖脱敏、切分泄露、标签泄露、挖掘判据这些内容。检测器本身也得做验证，所以我们用 mutate.py 给源码加可控的变异，确认每一项检测碰到对应的代码情况时真的能报警。变异测试得到两个结果：

1.新加的那条“检索不能使用 gold 派生信息”检测器，我们通过改代码变异（把 oracle 判断放宽到包含 all），确认它可以正确抓到问题。

2.变异测试还找出了一个覆盖缺陷：“切分保持会话完整”这个变异虽然被检测出来了，但靠的是血缘检查，并没有用用例直接验证切分逻辑。简单说就是“抓到问题了，但理由不对”。我们已经补上用例，专门校验时间顺序，不再只依赖 session_id 字母排序。

3.claims.json 里我们还特意留了两个红灯：C-255（召回率区间下限不满足）和 C-304（冲突判定区间下限不满足），状态保持“不通过”，一键复现脚本跑起来会一直报这两项。如果我们把标准放宽，或者换成点估计的方式，就等于让检查规则迁就我们想要的结论。

六、结论、局限与后续工作

这份报告给出了银河麒麟端侧 OS Agent 的一套方案，用来做多源融合偏好和知识记忆优化。赛题七个要求里，我们完成了(1)(2)(3)(4)，还给四项硬指标测了带置信区间的结果。方案有三个核心思路：知识来自事件之间的关联，事件本身不算知识；偏好准不准，

拿留出期真实行为判断，不靠人工标注；处理冲突时只给旧知识标上适用范围，不会直接覆盖旧内容。这三点全都在真实数据上跑通并完成测量。

两组独立对照实验（4.9、4.10 节）一起证明：效果来自这套机制本身。换不同生成模型，记忆机制在四档模型上都能带来正向收益，而且这个结论不受评判口径影响；换成不同向量模型，赛题指定接口和 BGE-M3 在这个任务上各有优劣，不存在明显短板。前者说明增益不是靠某个特定模型，后者说明也不是靠某个 embedding。

项目过程里总结出来的这套思路，和具体数字一样重要。立项时好几个预设的猜想，实测之后发现不对，我们完整记录了推翻猜想的全过程，没有偷偷修改结论。实测发现：常驻服务架构基本没收益；真正瓶颈是文本长度；官方向量引擎在当前数据量下并不比 numpy 更轻，内存反而高出 2.4 倍；安全策略的偏好信号问题，根源是单日批量数据污染再次出现；S3 冲突矿脉实测是空的；四条关联检索轴，在当前语料规模下都看不到统计上明显的正向提升。这些记录，加上我们特意保留的两个红灯指标，才是这套方案可信度真正的来源。

还有一条本身就是结论的负面结果：D1 - D7 采集按每天 16 条固定锚点任务再加 285 条生成任务全部跑完。但知识单元数量没能从 86 条涨到预期的约 119 条。原因在设计阶段就写清楚了：模板、偏好和冲突，看重的是数据重复度和变化幅度，不是单纯堆数据量。这两者不会靠多跑任务自动获得。采集确实把总量拉上去了，但重复度受限于只

有七天采集时间。这个负面结果，直接支撑了 4.8 节里 “单纯扩充语料没法提升召回率” 这个结论。

下面讲清楚已知局限和后续方向：计划内所有开发和实验都做完了，没有还在进行的任务。这部分只写现有工作的边界：哪些指标测了但是没达标，哪些功能做到哪一步。

（一）测过、但是不达标的指标

1.知识检索召回率：recall@10 单点值 86.3% 是达标的，但 Wilson 95% 置信区间下限只有 81.3%，不满足要求，claims.json 里 C-255 保持红灯。4.8 节已经量化说明：扩充语料不仅没用，还会拉低区间下限。唯一可行的办法是降低语料规模带来的负面影响，也就是让检索在数据变多的时候不掉分。这需要新的检索机制，不是更多数据。

2.偏好提取的风格类输出：提升幅度是 0，属于结构性问题。这类场景目前只找到一种触发情况，类别基线和这个唯一场景下的多数结果数学上本来就相等。后续思路是细化场景划分，比如按目录或者项目上下文区分。

3.知识冲突处理单独看留出集：合并样本 n=96 时达标，下限 88.4%；单独看留出集 n=60，下限 84.1%，还差 3.9 个百分点，C-304 保留红灯。丢分主要集中在 A 层数值漂移（7 例里占 5 例）。后续打算针对这一层细化判定规则，同时继续扩大留出集。

（二）功能实现范围的边界

1.BGE-M3 对照实验只做了基础 naive 版本。带元数据先验、MMR、BM25 的系统版本没有测。受时间限制，当时优先隔离向量模型本身带来的差异；而且系统层

后处理会冲淡 embedding 本身的影响，预估两组差距会更小。本次实验记录 M-211 以及 7 条结论 C-410~416 已经接入可复现体系，数据纳入机器校验。

2.横向能力：赛题要求 (5) 里的敏感信息识别过滤和自然语言指令驱动的精准遗忘已经做完，还有单独评测（分别见 3.2 节与 4.11 节）。剩下的：短中长期记忆流转机制（要求 (6)）只做到够用能用。

3.真实应用案例还没有单独成文。记忆能力封装成了 OS Agent (Pi Agent)能调用的 HTTP 接口，服务端、pi 注入钩子、启动脚本和两份验收脚本都已经提交到代码仓库（记忆服务 / 目录）。4.9 节那 104 组消融实验，就是靠这个服务切换“开 / 关记忆”跑出来的，那次实验也验证了整条链路可用。只是还没单独写一份真实场景案例文档。消融实验本身可以当作案例，但它是对照实验的写法，不是应用案例的写法。

七、安装部署指南

本章覆盖两套不同的环境，答题要求 (5)（安装部署指南、操作说明）对应的是前者：

端侧运行环境（**本章**）：**银河麒麟机器，记忆服务实际运行的地方，需要官方 KYSDK Embedding、官方向量引擎等系统级组件，安装过程涉及不可变系统的解锁。**

评测复现环境（**第八章 3 节**）：**任意 Windows / macOS / Linux 机器，只需 Python 与 numpy，用于核对本报告的数字主张，不涉及本章的系统级安装。**

两者不要混淆：复现评测数字不需要一台麒麟机；但要让记忆服务真正跑起来、能被 OS Agent 调用，必须完成本章的全部步骤。

1. 环境要求

表7.1 端侧运行环境要求

项目	要求
目标机	银河麒麟桌面操作系统 V11（磐石 OSTree 不可变系统），2 vCPU / 3.9 GB 起
系统级依赖	KYSDK Embedding SDK、`kylin-ai-runtime`、`kytensor`、官方向量引擎 `kylin-ai-vector-engine`——均通过麒麟软件源以 apt 安装，不在 `requirements.txt` 管理范围内
Python	3.9 及以上（`list[float]` 这类内建泛型标注，3.8 跑不了）
Python 第三方依赖	仅 `numpy>=1.24`（`requirements.txt` 唯一一项），执行 `pip install -r requirements.txt`

2. 系统解锁与 KYSDK 部署

磐石不可变系统对写入 `/usr` 设了两道防护：安装守卫拦截未经许可的 dpkg 操作，`/usr` 本身以只读方式挂载。首次在一台全新的麒麟机上部署，需按以下顺序完成：

查看当前部署状态：`sudo ostree admin status`，确认是干净的单一部署槽。

解锁 `/usr` 为可写：`sudo ostree admin unlock --hotfix`，随后用 `grep ' /usr ' /proc/mounts | tail -1` 验证结尾为 `rw`——`/usr` 常出现多行 overlay 记录，第一行恒为只读的中间层，必须取最后一行判断，只看第一行会误判为未解锁成功。

安装 embedding 开发包，走 `unshare -m` 通道以绕过安装守卫：`sudo apt update` 之后 `sudo unshare -m sh -c 'apt install -y libkylin-coreai-embedding-dev'`。

麒麟出厂镜像的后端版本通常落后于 SDK 要求的协议版本，需要安装 AI 子系统元包并协同升级。由于 dpkg 会把元包依赖判定为"已满足"而只重装元包自身、不碰其 71 个真实依赖，须显式取出完整依赖清单并逐一强制重装：先用 `apt-cache depends --recurse` 导出清单，再对清单中的每个包执行 `apt install -y --reinstall`。这一步同时会修复模型包 `model_description.yaml` 缺失 `others_info` 字段导致的 `errorCode 10`（模型未找到）问题。

重启后端服务使新协议生效：`sudo systemctl restart kytensor`；`kylin-ai-runtime` 由图形会话拉起、无 systemd 单元，需要在图形界面注销重新登录，或按第 5 小节所述方式手动拉起。

放行 kysec 执行控制以运行自编译的验证程序：`sudo setstatus -f exectl off`，编译并运行验证程序确认输出向量维度为 768，随后执行 `sudo setstatus -f exectl warning` 恢复安全策略。

完整命令、每一步的验证方法与已知报错的对照表，见《证明向量维度/前置步骤-不可变系统解锁与 KYSDK 部署命令手册.md》。

3. 一个必须先知道的限制：这套部署不会持久

以上安装写入的全部内容——`sshd`、embedding SDK 全套、`kytensor`、`kylin-ai-runtime`——在磐石不可变系统上属于临时覆盖层，**整机重启后必定退回出厂状态**，这一点已经过正常重启与异常断电两种

场景反复实证，虚拟机快照备份同样无法挽回，因为这些改动从未落到磁盘。这不是操作失误，是不可变系统"保护系统分区不被持久修改"这一设计目标的必然结果。

因此本项目把恢复流程固化为一键脚本，**每次整机重启后运行一次即可：**

```
bash ~/restore_env.sh
```

脚本自动完成上一节第 2 至 6 步的全部工作（解锁、修复依赖、重装 SDK 与模型包、重启后端服务、验证），约 5~10 分钟。若需要免人工干预，可进一步配置为开机自动执行——因为解锁与安装可以在无人登录的开机阶段完成，而 `kylin-ai-runtime` 必须挂在图形会话下才能启动，这部分被拆成两个脚本、由两种机制分别触发：`restore_env_boot.sh` 由 root 的 `@reboot` crontab 触发，负责系统层恢复；`start_runtime_session.sh` 由桌面自启动条目触发，负责在用户登录后拉起 `kylin-ai-runtime`。完整配置方法见前述手册第 7.4 节。

4. 构建知识索引

确认官方向量引擎已在运行（它由 systemd 用户级服务管理，需带 `--user` 查询）：

```
systemctl --user is-active kylin-ai-vector-engine   # 期望 active
```

随后构建生产索引：

```
python3 知识检索/build_knowledge_index.py --backend kylin --collection
rag_knowledge --exclude-conflicts
```

`--exclude-conflicts` 是复现本报告全部检索指标所必需的：不加此参数会把知识冲突记录也当作第四类知识单元一并入库，索引膨胀且不再与冻结评测集口径一致（历史上确实因遗漏此参数导致过一次索引膨胀 21% 的事故，详见《基准重建-执行方案.md》）。默认后端为 `numpy`，生产部署需显式指定 `--backend kylin`。构建成功后会打印知识单元的分类计数，应为案例、流程、模板三类合计 86 条。

5. 启动记忆服务

```
bash 记忆服务/start_memory_service.sh
```

默认以 `config=all backend=kylin` 启动，监听 `127.0.0.1:8077`。启动脚本会先自检 embedding 客户端能否返回 768 维向量，失败则提示先运行 `restore_env.sh`；随后以 `nohup` 常驻（磐石不可变系统没有可持久的 systemd 单元），并轮询 `/health` 接口直至服务就绪或超时。切换检索配置用环境变量：`CONFIG=naive bash 记忆服务/start_memory_service.sh`。

6. 验证安装

```
python3 记忆服务/smoke_service.py --config all
```

冒烟测试发出 5 条手工查询，全部应在 500 毫秒内返回、结构合法、`config` 字段与启动参数一致。看到全部通过，即完成从系统解锁到服务可用的全部安装部署。

八、操作说明

1. 知识检索接口

记忆服务运行后暴露两个 HTTP 接口：

表8.1 知识检索接口契约

接口	请求	响应
POST /retrieve	{"query": "任务描述文本", "k": 5}，k 取值 1~50，默认 5	{"units": [{"doc_id", "type", "text", "score"}, ...], "config": "all", "elapsed_ms": 74.3}
GET /health	无	{"ok": true, "index_size": 86, "config": "all", "backend": "kylin"}

config 字段在每次响应中回显，用于证明服务当前用的是哪一套检索配置；服务启动时若指定 --config meta_oracle 会直接拒绝启动，因为该档使用了题目标准答案自身的类型标签做过滤，属于标签泄漏、只是上界不是可部署的能力（3.6 节已说明）。score 字段仅供参考：对朴素检索它是排序依据本身，对带重排的配置它只是向量层相似度，不是最终排序依据，调用方不应把它当排名字段使用。

不需要先起服务、只想调试单条查询时，可直接用命令行工具：

```
python3 知识检索/search_knowledge.py "查询文本" --backend kylin --top-k 5
```

常用参数：--type 限定返回类型（case / workflow / template / conflict）；--mmr 启用多样性重排（--mmr-lambda 调节强度）；--hybrid 启用 BM25 混合检索（--hybrid-alpha 调节权重）；--include-superseded 关闭知识取代过滤、连同已被取代的旧版本一并返回。

2. 偏好查询与冲突回溯

图 3.1 架构图中"偏好查询""冲突回溯"两项能力，目前的实现方式与知识检索不同，如实说明：

偏好查询**是离线产物，不是运行时接口。偏好提取的结果落在** 知识检索**/preferences.jsonl，每条记录以 stable_key 标识某一情境下的稳定操作模式，按需直接读取该文件即可查询；运行时不重新计算。**

冲突回溯**是一个 Python 库函数，不是 HTTP 接口：mine_conflicts.retrieve_old(stable_key) 。传入某条知识的 stable_key，返回其旧版本记录；若查不到会直接抛出 KeyError，不会静默返回空值——这是有意的设计，避免"这条知识从未存在过"与"这次恰好没查到"在调用方看来无法区分。**

这两条能力目前均未封装成独立的 HTTP 端点。若后续需要通过网络调用，可复用 记忆服务/memory_service.py 现成的路由骨架各自新增一个 handler，当前版本尚未实现，如实标注为边界而非默认已具备。

3. 精准遗忘

三个接口（POST /forget 两步确认、POST /unforget、GET /forgotten）的完整契约见 3.7 节表3.4。调用示例：

```
# 第一步：定位候选
curl -s -X POST http://127.0.0.1:8077/forget \
     -d '{"instruction": "忘掉那条 sqlite3 找不到命令的处理记录", "k": 5}'

# 第二步：确认遗忘（doc_id 取自上一步候选列表）
curl -s -X POST http://127.0.0.1:8077/forget \
     -d '{"instruction": "忘掉那条 sqlite3 找不到命令的处理记录",
"confirm_doc_id": 12}'
```

4. 接入 OS Agent：pi 记忆钩子

记忆服务/rag-memory.ts 是 pi coding agent 的行动前记忆注入钩子，用 --memory on / --memory off 控制是否调用记忆服务：开启时，钩子在智能体开始行动前调用 POST /retrieve，把返回的知识单元拼装成注入块，插入到最后一条用户消息之前（系统提示之后、任务之前）；关闭时完全不调用该接口，其余提示词一字不变——两种条件下的系统提示逐字相同，这一点由钩子的实现方式保证，不是配置约定。这正是 4.9 节模型消融实验里"调 / 不调记忆"这一变量的实际实现。服务不可用时钩子会记录一条错误并降级为无记忆继续运行，不阻塞智能体。

5. 评测复现流程

对应答题要求中的"量化评测实施方案"：本报告全部量化结论按以下固定顺序产出与核对，任何人可按此顺序重新走一遍。

pip install -r requirements.txt——唯一第三方依赖 numpy。

cd 可复现性体系 && python reproduce_L1.py——纯 Python 重跑，任何机器可执行，无需麒麟环境，2~3 分钟内输出「合计 N 条主张：通过 M、不符 2、跳过 0」，与《当前数字看板.md》逐条核对。

需要麒麟真机的部分：确认 ~/restore_env.sh 已跑通、sudo setstatus -f exectl off 放行执行控制后，运行 python3 measure_L2.py，测量完毕执行 sudo setstatus -f exectl warning 恢复安全策略。

每次指标发生变化后，重跑 python 可复现性体系/生成数字看板.py 更新权威计数，再跑 python 可复现性体系/check_report_numbers.py 核对本报告正文里的数字与看板是否一致——退出码非零即表示有数字未同步。

变异测试自检（可选，验证"检测器本身有效"而非验证结论）：`python mutate.py --all`。

完整的操作细节、每步预期耗时与常见问题，见《准备复现.md》与《可复现性体系/可复现性说明.md》第四节；本节只给出固定顺序。

参考文献

[1] 麒麟软件有限公司，《XA-202612 OS Agent 记忆优化及高效应用研究 比赛方案》，2026 年。

[2] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[3] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[4] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in Proc. ACM SIGIR, pp. 335–336, 1998.

[5] J. Wang, X. Yi, R. Guo, et al., "Milvus: A Purpose-Built Vector Data Management System," in Proc. ACM SIGMOD, pp. 2614–2627, 2021.

[6] openKylin, "kylin-coreai-embedding — 文本向量化 SDK 调用示例," https://gitee.com/openkylin/kylin-coreai-embedding ；"libkysdk-vector-engine-client — 向量数据库 SDK 示例," https://gitee.com/openkylin/libkysdk-vector-engine-client 。

[7] J. Chen, S. Xiao, P. Zhang, et al., "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," arXiv:2402.03216, 2024.

[8] P. Budzianowski, T.-H. Wen, B.-H. Tseng, et al., "MultiWOZ — A Large-Scale Multi-Domain Wizard-of-Oz Dataset for Task-Oriented Dialogue Modelling," in Proc. EMNLP, 2018. arXiv:1810.00278

[9] S. Zhang, E. Dinan, J. Urbanek, et al., "Personalizing Dialogue Agents: I have a dog, do you have pets too?," in Proc. ACL, 2018. arXiv:1801.07243

[10] Y. Li, H. Su, X. Shen, et al., "DailyDialog: A Manually Labelled Multi-turn Dialogue Dataset," in Proc. IJCNLP, 2017.

[11] N. Craswell, B. Mitra, E. Yilmaz, et al., "Overview of the TREC 2019/2020 Deep Learning Track," https://trec.nist.gov/

[12] T. Nguyen, M. Rosenberg, X. Song, et al., "MS MARCO: A Human Generated MAchine Reading COmprehension Dataset," arXiv:1611.09268, 2016.

[13] Y. Qin, S. Liang, Y. Ye, et al., "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs," arXiv:2307.16789, 2023.（OpenBMB/ToolBench）

[14] Q. Xu, F. Hong, B. Li, et al., "On the Tool Manipulation Capability of Open-source Large Language Models," arXiv:2305.16504, 2023.（SambaNova/ToolBench）

[15] D. Sola, C. Warmuth, B. Schäfer, et al., "SAP Signavio Academic Models: A Large Process Model Dataset," arXiv:2208.12223, 2023.

[16] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," Journal of the American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927.

附录 A：交付物与代码位置对照

表 A.1 交付物与代码位置对照

模块	代码位置	说明报告
环境部署与 768 维验证	证明向量维度/	《证明向量维度-KYSDK-Embedding-768-代码执行报告.md》
端到端最小管线	端到端最小管线/	《说明报告.md》
向量化常驻服务与性能分档	向量化常驻服务/	《说明报告.md》
多源数据整合	多源数据整合/	《说明报告.md》
知识挖掘与检索	知识检索/	《说明报告.md》《向量引擎接入报告.md》《存储后端选型对比报告.md》
偏好提取	知识检索/mine_preferences.py	《偏好提取说明报告.md》
知识冲突处理	知识检索/mine_conflicts.py	《知识冲突处理说明报告.md》
评测集与召回率	评测集/	《召回率实测报告.md》《说明报告.md》
可复现性体系	可复现性体系/	《可复现性说明.md》《勘误与修正记录.md》
关联检索消融	评测集/measure_recall.py	《消融修复报告.md》
模型消融 Benchmark	评测集/模型消融/	《模型消融4x2-报告-v2.md》
BGE-M3 对照实验	BGE-M3对照/	《说明报告.md》《复现.md》《未完成事项.md》

续表 A.1

模块	代码位置	说明报告	
敏感信息过滤与脱敏评测	多源数据整合/anonymize.py、measure_anonymize_quality.py	《说明报告.md》	
记忆服务对外接口	记忆服务/（服务端 + pi 钩子 + 启动与验收脚本）	《说明报告.md》	
源代码脱敏与提交前扫描	脱敏/scan_identity.py	《脱敏说明.md》	《源码脱敏-实施方案.md》
精准遗忘	记忆服务/forget_ledger.py、记忆服务/遗忘评测/	《说明报告.md》	《预登记.md》

附录 B：初版待更新数字的结清说明

初版（v0.1）用专门的记号标出了 7 组会随 D2–D7 采集变动的数字，并承诺采集收工后重测。本附录逐条说明结清结果。定稿全文已撤掉这些记号，因为每一条都已有确定去向。

表 B.1 初版待更新数字的结清说明

编号	涉及数字	初版值	结清结果
B-1	数据集记录数	9,807 条	维持。采集新增的是麒麟本机会话，未并入本报告的建索引语料
B-2	知识单元总数（表3.2）	86 条（31/54/1）	维持 86 条。预期的增长没有发生，原因见第六章
B-3	召回率全表（表4.2、表4.3）	R@10 = 86.3%	维持。语料未变，评测集 227 题冻结未动
B-4	消融七配置（表4.4）	all = 86.8%	维持。关联检索消融阶段二据「语料未增长」判定 86 条即最终口径
B-5	偏好三类准确率（表4.6）	操作习惯 90.0%	维持
B-6	冲突判定（表4.7）	97.2%（n=36 开发集）	已更新。2026-09-07 完成留出集单人全量盲测，改为三口径并报：开发集 97.2% / 留出集 93.3% / 合并 94.8%（下界 88.4%，达标）
B-7	硬指标汇总（表4.12）	见表	随 B-6 更新，冲突一项由「点估计过线、下界不过」变为达标

结清原则：B-1~B-5 维持不是「没来得及重测」，而是重测的前提（语料增长）经实测未发生；B-6 是唯一发生实质变动的一项，且变动

方向对本方案有利，因此在正文中同时保留了不利的留出集单独口径，两个口径并读。

定稿新增的数字（4.9 节 模型消融、4.10 节 BGE-M3 对照）不带待重测记号：两个实验均已跑完并冻结，其未闭合部分是范围上的（系统档未做、claims 未接入），不是数值上的待重测。